

raft-java Developer Cheatsheet

Felix Berger

June 2026

Quick Start

```
./gradlew build
```

```
java -jar build/libs/raft-java-1.0.0-all.jar \
  config/node1.properties
```

Configuration (.properties)

```
node.id=node1
node.port=9091
data.dir=/var/raft/node1
peer.node1=host1:9091
peer.node2=host2:9092
peer.node3=host3:9093
```

```
snapshot.threshold=100
snapshot.chunk.size=1048576
metrics.port=0
```

```
rpc.timeout.request.vote.ms=1000
rpc.timeout.append.entries.ms=2000
rpc.timeout.install.snapshot.ms=30000
rpc.timeout.pre.vote.ms=1000
```

Custom State Machine

```
public class MyStateMachine
    implements StateMachine {

    @Override
    public byte[] apply(byte[] command) {
        return "OK".getBytes();
    }

    @Override
    public byte[] takeSnapshot() {
        return serialize(state);
    }

    @Override
    public void restoreSnapshot(byte[] data) {
        state = deserialize(data);
    }
}
```

Register as a Spring bean:

```

@Bean
public StateMachine stateMachine() {
    return new MyStateMachine();
}

```

Submitting Commands

In-process (server-side):

```
RaftNode node = ctx.getBean(RaftNode.class);
```

```

CompletableFuture<byte[]> f =
    node.submitCommand(cmd);
byte[] result = f.get(2, SECONDS);

```

Over gRPC (client-side):

```

RaftClient client = new RaftClient(
    Map.of("n1", "host1:9091",
           "n2", "host2:9092",
           "n3", "host3:9093"));
byte[] result = client.submit(cmd);

```

Leadership Transfer (new)

```

node.transferLeadership("n2")
    .get(1, SECONDS);

```

Leader stops accepting commands, catches up the target, sends TimeoutNow. Aborts automatically on timeout.

Cluster Reconfiguration (new)

```

node.addServer("n4", "host4:9094")
    .get(2, SECONDS);

```

```

node.removeServer("n4")
    .get(2, SECONDS);

```

One change at a time. Rejected until leader's no-op commits (errata fix).

Transport Layer

Switch transport (instead of default gRPC):

```

@Bean
public RaftTransportFactory transportFactory() {
    return new NettyTransportFactory();
    // or: new HadoopTransportFactory();
}

```

Custom transport – implement RaftTransport:

```

public class MyTransport
    implements RaftTransport {

    CompletableFuture<RequestVoteResponse>
        requestVote(RequestVoteRequest req);
    CompletableFuture<AppendEntriesResponse>
        appendEntries(AppendEntriesRequest req);
    CompletableFuture<InstallSnapshotResponse>
        installSnapshot(InstallSnapshotRequest r);
    CompletableFuture<PreVoteResponse>

```

```

    preVote(PreVoteRequest req);
    CompletableFuture<TimeoutNowResponse>
        timeoutNow(TimeoutNowRequest req);
    void close();
}

```

Available Transports

Transport	Protocol	Bean class
gRPC	HTTP/2 + protobuf	GrpcTransportFactory
Netty	TCP + len-delimited	NettyTransportFactory
Hadoop	WritableRpcEngine	HadoopTransportFactory

All transports wrapped with `TimeoutTransport` automatically. gRPC and Netty support TLS/mTLS via `TlsConfig`.

Storage Implementations

Class	Durability	Use
BerkeleyDbStorage	fsync	Production default
WalStorage	Append-only WAL	Lightweight alternative
InMemoryStorage	None	Tests only

```

@Bean
public RaftStorage raftStorage(RaftConfig cfg) {
    return new WalStorage(cfg.dataDir().toFile());
}

```

Key Interfaces

Interface	Purpose
StateMachine	Your app logic
RaftStorage	Durable state
RaftTransport	Peer comms (5 RPCs)
RaftTransportFactory	Connection factory
RaftRpcHandler	Incoming RPCs
RpcTimeouts	Per-RPC timeouts
TlsConfig	TLS/mTLS settings

Node Lifecycle

```

System.setProperty("raft.config.path",
    "config/node1.properties");
var ctx = new AnnotationConfigApplicationContext(
    RaftNodeConfiguration.class);
RaftNode node = ctx.getBean(RaftNode.class);
node.start();
ctx.close();

// Manual (tests)
RaftNode node = new RaftNode(
    config, storage, stateMachine,

```

```

    transportFactory, RaftMetrics.noop());
node.start();
node.shutdown();

```

Testing

```

RaftNode node = new RaftNode(cfg,
    new InMemoryStorage(),
    new KeyValueStateMachine(),
    addr -> null,
    RaftMetrics.noop());
node.start();

// In-process gRPC (multi-node)
RaftTransport t = new GrpcTransport(
    InProcessChannelBuilder
        .forName(addr).directExecutor()
        .build());

./gradlew test
./gradlew jacocoTestReport

```

Logging

SLF4J + Logback. Config: src/main/resources/logback.xml

Logger	Default level
com.fbtberger.raft	INFO
io.grpc	WARN
io.netty	WARN
com.sleepycat	WARN

Proto RPCs (raft.proto)

RPC	Direction	Purpose
RequestVote	peer	Election
PreVote	peer	Pre-election
AppendEntries	leader	Replication
InstallSnapshot	leader	Catch-up
TimeoutNow	leader	Transfer

Client RPCs: Submit, AddServer, RemoveServer

Health Checks

```

curl http://localhost:10091/health # liveness (always 200)
curl http://localhost:10091/ready  # readiness (200/503)

```

JMX

MBean: com.fbtberger.raft:type=RaftNode
 Attributes: role, term, commitIndex, clusterMembers, transferInProgress
 Operations: triggerSnapshot(), transferLeadership(targetId)
 All Micrometer metrics also published to JMX.

Metrics (Prometheus + JMX)

`curl http://localhost:10091/metrics`

Counters: `raft.election.started`, `.won` | `raft.vote.granted` | `raft.stepdown` | `raft.entry.applied` | `raft.replication.sent`, `.success`, `.failure` | `raft.snapshot.taken`, `.installed`, `.chunk.sent`, `.chunk.received` | `raft.client.rejected`

Timers: `raft.rpc.append.entries`, `.request.vote`, `.install.snapshot` | `raft.client.submit`

Gauges: `raft.node.term`, `.commit.index`, `.last.applied`, `.role`, `.log.last.index`, `.snapshot.index` | `raft.cluster.size`

Performance Tuning

Parameter	Default	Effect
<code>snapshot.threshold</code>	100	Entries before auto-snapshot
<code>snapshot.chunk.size</code>	1 MB	InstallSnapshot chunk size
<code>rpc.timeout.*.ms</code>	1-30s	Per-RPC timeout
<code>HEARTBEAT_INTERVAL_MS</code>	50	Heartbeat frequency
<code>ELECTION_TIMEOUT_*_MS</code>	150-300	Election timeout range
<code>MAX_BATCH_BYTES</code>	1 MB	Max AppendEntries size
<code>MAX_INFLIGHT_APPENDS</code>	2	Pipeline depth per peer